BST vs Hash Table

I implemented a binary search tree (unbalanced, no duplicates) and a hash table with separate chaining (capacity 16, doubled when the load factor exceeds 0.75) for int keys. I timed insert, search and delete over all n keys for n = 100, 1000 and 10000, on random and sorted input. Values are the median of 3 runs.

Complexity

In BST each operation costs O(h), where h is the height. Best case O(1) (key at the root), average O(log n) on random input, worst case O(n) when sorted input makes the tree a linked list. Hash tables best and average O(1), worst case O(n) if all keys land in one bucket. A resize costs O(n), but insert is O(1) amortized.

Results (microseconds, BST / Hash)

Random keys (BST height 13 / 26 / 29)

Operation	n = 100	n = 1000	n = 10000
insert	3933 / 322	1341 / 1465	6956 / 9960
search	92 / 54	1327 / 161	12499 / 1979
delete	107 / 65	1689 / 979	10510 / 2009

Sorted keys (BST height 99 / 999 / 9999)

Operation	n = 100	n = 1000	n = 10000
insert	52 / 97	5825 / 205	330114 / 9342
search	42 / 16	2813 / 139	723808 / 2206
delete	11 / 13	116 / 156	1524 / 1392

Findings

1. On sorted input the BST height is n-1, so it is a linked list. When n grows 10 times, BST search grows from 2813 to 723808 us, which is quadratic over the series (O(n) per operation). The hash table grows about 10 times (O(1) per operation).

2. On random input the BST height stays near log n (29 for 10000 keys), so both are close. Hash search is about 2-6 times faster. Insert is about equal, because my hash insert calls contains first and the table resizes about 10 times.

3. BST delete on sorted input is fast only because keys are deleted in ascending order (always the root without a left child). It does not change the O(n) worst case.

4. The hash chain length was 1 because keys are 0..n-1 and spread evenly, so the hash worst case was not tested.

Why practice differs from theory

The BST depends on insertion order, the hash table on the hash function and load factor. A hash lookup is one index computation and a short chain, while a BST follows pointers through many separate objects (more cache misses). Results at n = 100 are noisy because of JIT, class loading and GC, so I used medians and compared trends. The BST keeps keys ordered (min, max, inorder), the hash table does not.